

Hafta 2 — Kapsülleme (Encapsulation)

Öğrenci Yönergesi — Adım Adım Uygulama

Bilgi	Değer
Ders	Nesne Tabanlı Programlama (2. Sınıf)
Konu	Kapsülleme (Encapsulation)
Ön koşul	Hafta 1 — sınıf ve nesne
Ortam	Visual Studio Community 2026 · .NET 10 (LTS)
Dil	C#
Süre	Yaklaşık 2–3 saat
Teslim	Çalışan proje klasörü (.zip)

1. Bu Haftanın Hedefi

Bu hafta nesnelerin verisini dış müdahaleden nasıl koruyacağımızı öğreneceğiz. Bir sınıfın içindeki değerlere doğrudan erişilmesini engelleyip, bu değerlerin yalnızca belirlediğimiz kurallara uygun biçimde değiştirilmesini sağlayacağız. Buna kapsülleme (encapsulation) denir ve nesne tabanlı programlamanın temel taşlarından biridir.

2. Kazanımlar

Bu haftanın sonunda şunları yapabilir hâle geleceksiniz:

- Erişim belirleyicileri (public, private) doğru kullanmak,
- property (get/set) tanımlamak ve set içinde doğrulama yapmak,
- auto-property ve salt-okunur property arasındaki farkı açıklamak,
- constructor (yapıcı metot) yazarak nesneyi tutarlı başlangıç değerleriyle oluşturmak,
- veriyi koruyan, kurallı çalışan bir sınıf tasarlamak.

3. Hatırlatma: Geçen Hafta Nerede Kalmıştık?

Geçen hafta Öğrenci sınıfındaki tüm alanları public yapmıştık. Bu yüzden dışarıdan şu satır hiçbir engelle karşılaşmadan çalışıyordu:

```
ogr.Vize = -50; // negatif not! ama kimse engellemiyor
```

Bu bir sorundur: verinin geçersiz duruma düşmesini kimse önlemiyor. Bu hafta bu açığı kapatacağız.

4. Temel Kavramlar

4.1. Erişim Belirleyiciler

- public:** Sınıfın dışından da erişilebilir.
- private:** Yalnızca sınıfın kendi içinden erişilebilir. Alanları (field) private yapmak, kapsüllemenin ilk adımıdır.

4.2. Property (Özellik)

Bir field'ı doğrudan public yapmak yerine, ona kontrollü erişim veren yapıdır. get bloğu değeri okur, set bloğu yeni değeri atar. set içinde value, dışarıdan gelen değeri temsil eder; doğrulama tam da burada yapılır.

```
private int yas;  
public int Yas  
{  
    get { return yas; }  
    set  
    {  
        if (value >= 0)    // gelen değer (value) kontrol ediliyor  
            yas = value;  
    }  
}
```

4.3. Auto-Property (Kısa Yazım)

Doğrulama gerekmiyorsa property'nin kısa hâli kullanılır. Arka planda gizli bir field otomatik oluşturulur:

```
public string Ad { get; set; }
```

4.4. Salt-Okunur Property

Yalnızca get içeren bir property dışarıdan okunabilir ama değiştirilemez. Böylece bir değer görünür olur ama yalnızca sınıfın metotlarıyla değişir:

```
public double Bakiye { get { return bakiye; } }
```

4.5. Constructor (Yapıcı Metot)

Nesne new ile oluşturulurken otomatik çalışan özel bir metottur. Sınıfla aynı adı taşır, dönüş tipi yoktur ve başlangıç değerlerini atar. Böylece nesne daha doğar doğmaz tutarlı olur.

this anahtar sözcüğü

this, o anki nesneyi işaret eder. Genellikle parametre adı ile alan/özellik adı aynı olduğunda ayırmak için kullanılır:

```
this.Ad = ad;
```

5. Uygulama: Banka Hesabı

Bakiyesi dışarıdan doğrudan değiştirilemeyen, para yatırma/çekme işlemlerini kurallarla yapan bir Hesap sınıfı yazacağız.

Önce projeyi oluşturun: **Hafta2_BankaHesabi** adında bir Konsol Uygulaması (C#, .NET 10). Sonra sağ tık → Ekle → Sınıf ile **Hesap.cs** dosyasını ekleyin.

Adım A — Hesap sınıfı ve korunan bakiye

bakiye alanını private yapıyoruz; dışarıya yalnızca salt-okunur bir property ile gösteriyoruz. SahipAdi için auto-property yeterli.

```
namespace Hafta2_BankaHesabi  
{  
    class Hesap  
    {  
        private double bakiye;    // dışarıdan doğrudan erişilemez  
  
        public string HesapNo { get; }    // salt-okunur (constructor'da atanır)  
        public string SahipAdi { get; set; } // auto-property  
  
        public double Bakiye    // salt-okunur property  
    }  
}
```

```
        {  
            get { return bakiye; }  
        }  
    }  
}
```

Adım B — Constructor ekleyin

Sınıfın içine, hesabı başlangıç değerleriyle oluşturan yapıcı metodu ekleyin. Negatif başlangıç bakiyesini de burada engelliyoruz:

```
public Hesap(string sahipAdi, string hesapNo, double baslangicBakiye)  
{  
    SahipAdi = sahipAdi;  
    HesapNo = hesapNo;  
    bakiye = baslangicBakiye >= 0 ? baslangicBakiye : 0;  
}
```

Adım C — Para yatırma ve çekme metotları

Bakiye yalnızca bu metotlarla, kurallara uyularak değişir. Geçersiz işlemler engellenir:

```
public void ParaYatir(double tutar)  
{  
    if (tutar <= 0)  
    {  
        Console.WriteLine("Yatırılacak tutar pozitif olmalı!");  
        return;  
    }  
    bakiye += tutar;  
    Console.WriteLine($"{tutar} TL yatırıldı. Yeni bakiye: {bakiye} TL");  
}  
  
public void ParaCek(double tutar)  
{  
    if (tutar <= 0)  
    {  
        Console.WriteLine("Çekilecek tutar pozitif olmalı!");  
        return;  
    }  
    if (tutar > bakiye)  
    {  
        Console.WriteLine("Yetersiz bakiye!");  
        return;  
    }  
    bakiye -= tutar;  
    Console.WriteLine($"{tutar} TL çekildi. Yeni bakiye: {bakiye} TL");  
}
```

Adım D — Main içinde test edin

```
using System;  
  
namespace Hafta2_BankaHesabi  
{  
    class Program  
    {  
        static void Main(string[] args)  
        {  
            Hesap hesap = new Hesap("Ayşe Yılmaz", "TR1001", 500);  
  
            Console.WriteLine($"Hesap sahibi: {hesap.SahipAdi}");  
            Console.WriteLine($"Başlangıç bakiyesi: {hesap.Bakiye} TL");  
  
            hesap.ParaYatir(250);  
            hesap.ParaCek(1000); // yetersiz bakiye -> engellenir  
        }  
    }  
}
```

```
hesap.ParaCek(300);

// hesap.bakiye = 1000000; // <-- bu satır HATA verir (bakiye private)

Console.WriteLine($"Son bakiye: {hesap.Bakiye} TL");
}
}
```

❗ Ne görmelisiniz?

Hesap sahibi: Ayşe Yılmaz
Başlangıç bakiyesi: 500 TL
250 TL yatırıldı. Yeni bakiye: 750 TL
Yetersiz bakiye!
300 TL çekildi. Yeni bakiye: 450 TL
Son bakiye: 450 TL

⚠️ Kapsüllemenin kanıtı

Main içindeki yorumlu satırın (hesap.bakiye = 1000000;) başındaki // işaretini kaldırıp deneyin: program artık derlenmez.
Çünkü bakiye private'tır ve dışarıdan değiştirilemez. Bakiye yalnızca ParaYatir/ParaCek ile, kurallara uyularak değişir. İşte kapsülleme budur.

6. Kendi Göreviniz (Lab Teslimi)

Yukarıdaki uygulamayı aşağıdaki gibi genişletin. Teslim edilecek çalışma budur:

- Hesap sınıfına doğrulama yapan bir property ekleyin: negatif değer kabul etmeyen bir FaizOrani (get/set, set içinde kontrol).
- İkinci bir hesap oluşturun ve iki hesapta da para yatırma/çekme işlemleri deneyin.
- Geçersiz bir işlem (örn. eksi tutar yatırma veya bakiyeden fazla çekme) deneyip uyarının çalıştığını gösterin.
- Her iki hesabın son bakiyesini ekrana yazdırın.

7. Sık Yapılan Hatalar

- Field'ı public bırakmak:** bakiye public olursa kapsülleme yok olur. Alanları private yapın.
- Property içinde kendini çağırmak:** get { return Bakiye; } yazmak sonsuz döngüye yol açar. Property, arka plandaki private field'ı döndürmelidir (return bakiye;).
- Constructor adını yanlış yazmak:** Yapıcı metodun adı sınıf adıyla birebir aynı olmalı ve dönüş tipi (void dahil) yazılmamalıdır.
- Doğrulamayı unutmak:** set veya metot içinde kontrol koymazsanız geçersiz değerler yine girer; kapsüllemenin amacı bu kontrolü koymaktır.

8. Değerlendirme

Ölçüt	Puan
Proje derleniyor ve çalışıyor	15
bakiye private ve salt-okunur property ile sunulmuş	25
Constructor doğru tanımlanmış	20

Ölçüt	Puan
ParaYatir / ParaCek doğrulama ile çalışıyor	25
Doğrulamalı property (FaizOrani) eklenmiş	15
TOPLAM	100

9. Ödev (Gelecek Haftaya Hazırlık)

Hesap sınıfına, başka bir hesaba para gönderen bir metot ekleyin:

```
public void ParaTransferi(Hesap hedefHesap, double tutar)
```

Bu metot kendi bakiyesinden tutarı çekmeli (yetersizse uyarı vermeli) ve hedef hesaba yatırmalıdır. İki hesap oluşturup transferi test edin.

Dikkat ettiyseniz bu metot, **bir nesnenin başka bir nesneyle etkileşmesini** sağlıyor. İşte gelecek haftanın konusu tam olarak bu: nesneler arası ilişkiler (composition).